Poll and Interrupt

In **polling**, CPU **steadily checks** the status of the port at **fixed intervals** to determine if it needs attention. use software to check it.

- Slow - need to explicitly check to see if switch is pressed
- Wasteful of CPU time - the faster a response we need, the more often we need to check.
- Scales badly - difficult to build system with many activities which can respond quickly. Response time depends on all other processing.

In **interrupt**, the device **informs** the CPU that it requires its attention.

- Fast - hardware mechanism
- Efficient - code runs only when necessary
- Scales well
- * ISR response time doesn't depend on most other processing.
- * Code modules can be developed independently

Hardware complexity:
Polling is simpler and less complex than interrupts.

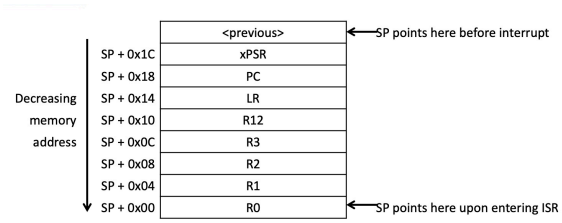
Responsiveness:
Interrupts are more responsive than polling because they avoid unnecessary checks and allow the CPU to perform other tasks.

Processing power:
Polling uses more power because the CPU is continuously checking for events. Interrupts use less power because the processor only executes an action when an interrupt is generated.

CPU's Hardwired Exception Processing
Enter:

1. **Finish** current instruction (except for lengthy instructions)
2. **Push** context (8 32-bit words) onto current stack (MSP or PSP).
   context: xPSR, PC(Return address), LR (R14), R12, R3, R2, R1, R0
3. **Switch** to handler (privileged) mode and use **MSP**
4. Load **PC** with address of exception handler
5. Load **LR** with **EXC_RETURN** code
6. Load **IPSR** with exception number

7*. Start **executing** code of exception handler
8*. Usually 16 cycles from exception request to execution of first instruction in handler



Exiting an Exception Handler:
1. **Execute** instruction triggering exception return processing
2. Select **return stack**, **restore** context from that stack
3. **Resume** execution of code at restored address

Eg: Explain how the duration of the interrupt is measured:
1. Toggle a GPIO Pin:
   • Set a GPIO pin HIGH at the start of the ISR and LOW at the end.
2. Measure Signal Pulse:
   • Use an oscilloscope or logic analyzer to measure the duration of the HIGH signal on the pin.
3. Determine Interrupt Duration:
   • The width of the HIGH signal corresponds to the time spent in the ISR.

Eg: what are the main considerations when designing a function to deal with an interrupt request in a control system?
Minimize Interrupt Latency
Keep ISRs Short and Efficient
Avoid Blocking or Waiting Operations